Web Security Assessment Lab: Guided Projects

Guided projects with real deliverables.

All projects run against deliberately vulnerable apps you host yourself (OWASP Juice Shop, DVWA, WebGoat, crAPI) or platforms that grant permission (PortSwigger Web Security Academy, Hack The Box, TryHackMe). Never test systems you don't own or lack written authorization for.

Lab setup (Project 0)

Goal: Build an isolated, repeatable testing environment.

- Run targets in Docker on a host-only or NAT network (e.g., `docker run -p 3000:3000 bkimminich/juice-shop`)
- Install Burp Suite Community or OWASP ZAP, plus a browser profile proxied through it
- Keep a note-taking system (Obsidian, CherryTree) and a findings template

Deliverables: Network diagram, setup README, snapshot of the working environment, blank report template.

Project 1: Recon and attack surface mapping

Target: Juice Shop

Tasks: Crawl the app, enumerate routes, parameters, roles, and technologies, review JS files for hidden endpoints, and document trust boundaries.

Deliverables: Attack surface inventory (spreadsheet), sitemap/endpoint list, data flow diagram, prioritized test plan.

Project 2: Authentication and session management review

Target: DVWA and Juice Shop

Tasks: Test login flows, password reset logic, session token entropy and flags (HttpOnly, Secure, SameSite), session fixation, and MFA/lockout behavior.

Deliverables: Findings mapped to OWASP ASVS V2/V3, evidence screenshots, remediation guidance (e.g., rate limiting, token rotation).

Project 3: Injection and input handling

Target: DVWA, WebGoat

Tasks: Identify SQL injection, XSS (reflected, stored, DOM), and command injection points at low security levels, then compare behavior at higher levels. Review the source to see why each fix works.

Deliverables: Vulnerability write-ups with root cause, CVSS scores, and secure code fixes (parameterized queries, output encoding, CSP header proposal).

Project 4: Access control and business logic

Target: Juice Shop, WebGoat

Tasks: Test IDOR, privilege escalation, forced browsing, and workflow bypasses (e.g., basket/coupon manipulation) using two or more user roles.

Deliverables: Access control matrix (role vs. function), proof-of-concept steps in Burp Repeater, recommended authorization model.

Project 5: API security assessment

Target: OWASP crAPI

Tasks: Map the API from the OpenAPI spec or proxy traffic, then test against the OWASP API Top 10: BOLA, broken auth, excessive data exposure, mass assignment, rate limiting.

Deliverables: Postman collection of test cases, API findings report, hardening checklist.

Project 6: Configuration, headers, and dependency review

Target: Any self-hosted app

Tasks: Audit TLS config, security headers, CORS, error handling, and exposed files. Run dependency scanning (OWASP Dependency-Check, `npm audit`) and compare automated results to manual findings.

Deliverables: Configuration baseline, scanner-vs-manual comparison with false-positive analysis.

Capstone: Full assessment and report

Tasks: Pick one app, define scope and rules of engagement, run a full methodology (OWASP WSTG), then retest after applying fixes.

Deliverables:

- Rules of engagement and scope document
- Executive summary (1 page, non-technical)
- Technical findings with severity, evidence, impact, and remediation
- Retest results showing fixed vs. open issues
- Short presentation (5-10 slides)

Skills you'll demonstrate

Methodology (WSTG/ASVS), proxy-based testing, risk rating (CVSS), secure code review, and professional reporting, which makes a strong portfolio.

Suggested pacing

Weeks	Projects
1	0 and 1
2-3	2 and 3
4-5	4 and 5
6	6
7-8	Capstone